

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЁТ
по лабораторной работе № 5
по курсу «Разработка кроссплатформенных приложений»
Вариант 4

Выполнили
студенты группы 22ВОЭ1
Брюзгин А. С.
Тихонов Д. А.

Приняли
Юрова О. В.

Пенза 2025

Цель работы

научиться создавать многопоточные приложения с использованием стандартных средств языка Java.

Задание

Модифицировать приложение из предыдущей лабораторной работы, реализовав вычисление определенного интеграла в нескольких дополнительных потоках (число потоков определяется номером варианта), снимая нагрузку с основного потока и предотвращая "подвисание" графического интерфейса

Исходный код программы

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */
package my.contacteditor;

import java.util.List;
import static java.lang.Math.tan;
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.atomic.AtomicInteger;
import javax.swing.JOptionPane;
import javax.swing.SwingUtilities;
import javax.swing.SwingWorker;
import javax.swing.table.DefaultTableModel;
import java.util.concurrent.*;

/**
```

```

*
* @author User
*/

public class ContactEditorUI extends javax.swing.JFrame {

    private LinkedList <RecIntegral> records = new LinkedList();

    private DefaultTableModel tModel;

    int i = 0;
    /**
     * Creates new form ContactEditorUI
     */
    public ContactEditorUI() {
        initComponents();
    }

    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">
    private void initComponents() {

        jTextField1 = new javax.swing.JTextField();
        jTextField2 = new javax.swing.JTextField();
        jTextField3 = new javax.swing.JTextField();
        jLabel1 = new javax.swing.JLabel();
        jLabel2 = new javax.swing.JLabel();
        jLabel3 = new javax.swing.JLabel();
        jScrollPane1 = new javax.swing.JScrollPane();
        jTable1 = new javax.swing.JTable();
        jButton1 = new javax.swing.JButton();
        jButton2 = new javax.swing.JButton();
        jButton3 = new javax.swing.JButton();
        jButton4 = new javax.swing.JButton();
        jButton5 = new javax.swing.JButton();

        setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);

```

```

jTextField1.setText("0");

jTextField2.setText("0");

jTextField3.setText("0");

jLabel1.setText("upper_limit");

jLabel2.setText("lower_limit");

jLabel3.setText("Step");

jTable1.setModel(new javax.swing.table.DefaultTableModel(
    new Object [][] {

        },
    new String [] {
        "Upper_limit", "Lower_limit", "Step", "Result"
    }
) {
    Class[] types = new Class [] {
        java.lang.Double.class, java.lang.Double.class, java.lang.Double.class, java.lang.Double.class
    };
    boolean[] canEdit = new boolean [] {
        false, false, false, false
    };

    public Class getColumnClass(int columnIndex) {
        return types [columnIndex];
    }

    public boolean isCellEditable(int rowIndex, int columnIndex) {
        return canEdit [columnIndex];
    }
});
jTable1.setRowHeight(30);
jScrollPane1.setViewportViewView(jTable1);
if (jTable1.getColumnModel().getColumnCount() > 0) {
    jTable1.getColumnModel().getColumn(0).setResizable(false);
    jTable1.getColumnModel().getColumn(1).setResizable(false);
    jTable1.getColumnModel().getColumn(2).setResizable(false);
    jTable1.getColumnModel().getColumn(3).setResizable(false);
}

```



```

        .addComponent(jScrollPane1, javax.swing.GroupLayout.Alignment.TRAILING,
javax.swing.GroupLayout.DEFAULT_SIZE, 477, Short.MAX_VALUE)
        .addGroup(javax.swing.GroupLayout.Alignment.TRAILING, layout.createSequentialGroup()
        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addComponent(jLabel1)
        .addComponent(jTextField1, javax.swing.GroupLayout.PREFERRED_SIZE, 122,
javax.swing.GroupLayout.PREFERRED_SIZE))
        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED, 53,
Short.MAX_VALUE)
        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addComponent(jTextField2, javax.swing.GroupLayout.PREFERRED_SIZE, 122,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addComponent(jLabel2))
        .addGap(58, 58, 58)
        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addComponent(jLabel3)
        .addComponent(jTextField3, javax.swing.GroupLayout.PREFERRED_SIZE, 122,
javax.swing.GroupLayout.PREFERRED_SIZE)))
        .addGroup(layout.createSequentialGroup()
        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.TRAILING, false)
        .addComponent(jButton4, javax.swing.GroupLayout.Alignment.LEADING,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        .addComponent(jButton1, javax.swing.GroupLayout.Alignment.LEADING,
javax.swing.GroupLayout.DEFAULT_SIZE, 122, Short.MAX_VALUE))
        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        .addComponent(jButton2, javax.swing.GroupLayout.PREFERRED_SIZE, 122,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addGap(54, 54, 54)
        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING, false)
        .addComponent(jButton5, javax.swing.GroupLayout.DEFAULT_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
        .addComponent(jButton3, javax.swing.GroupLayout.DEFAULT_SIZE, 122,
Short.MAX_VALUE)))
        .addContainerGap())
    );
    layout.setVerticalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
        .addGroup(layout.createSequentialGroup()
        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
        .addComponent(jLabel1)
        .addComponent(jLabel2)
        .addComponent(jLabel3))
        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)

```

```

        .addComponent(jTextField1, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addComponent(jTextField2, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE)
        .addComponent(jTextField3, javax.swing.GroupLayout.PREFERRED_SIZE,
javax.swing.GroupLayout.DEFAULT_SIZE, javax.swing.GroupLayout.PREFERRED_SIZE))
        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
        .addComponent(jScrollPane1, javax.swing.GroupLayout.PREFERRED_SIZE, 143,
javax.swing.GroupLayout.PREFERRED_SIZE)
        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.RELATED)
        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
            .addComponent(jButton1)
            .addComponent(jButton2)
            .addComponent(jButton3))
        .addPreferredGap(javax.swing.LayoutStyle.ComponentPlacement.UNRELATED)
        .addGroup(layout.createParallelGroup(javax.swing.GroupLayout.Alignment.BASELINE)
            .addComponent(jButton4)
            .addComponent(jButton5))
        .addContainerGap(javax.swing.GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE))
    );

```

```

    pack();
} // </editor-fold>

```

```

private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {

    Double ul = Double.valueOf(jTextField1.getText());
    Double ll = Double.valueOf(jTextField2.getText());
    Double stp = Double.valueOf(jTextField3.getText());

    try{
        records.add(new RecIntegral(ul,ll,stp));
        tModel = (DefaultTableModel) jTable1.getModel();
        tModel.addRow(new Object[] {ul, ll, stp});
        i++;
    }

    catch (ErrRecIntegralVal e) {

        JOptionPane.showMessageDialog(null, e.getMessage(), "Ошибка ввода",
JOptionPane.WARNING_MESSAGE);
        throw new RuntimeException("Error" + e.getMessage(), e);

    }

}

```

```

private void jButton3ActionPerformed(java.awt.event.ActionEvent evt) {

    tModel = (DefaultTableModel) jTable1.getModel();

    int rowNum = jTable1.getSelectedRow();

    if (rowNum == -1){
        JOptionPane.showMessageDialog(null, "Выберите строку для удаления");
    }else{
        tModel.removeRow(rowNum);
        records.remove(rowNum);
        i--;
    }
}

private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {

    long startTime = System.nanoTime();

    records.clear(); // Очищаем список записей

    AtomicInteger taskCounter = new AtomicInteger(i); // Счетчик оставшихся задач

    for (int j = 0; j < i; j++) {
        final int row = j; // Захватываем текущую строку для использования в потоке

        // Запускаем асинхронную задачу для каждой строки
        CompletableFuture.runAsync(() -> {
            try {
                Double ul = (Double) jTable1.getValueAt(row, 0);
                Double ll = (Double) jTable1.getValueAt(row, 1);
                Double stp = (Double) jTable1.getValueAt(row, 2);

                Double rez = calculateIntegral(ll, ul, stp); // Вычисляем интеграл

                // Обновляем таблицу и добавляем запись в records
                SwingUtilities.invokeLater(() -> {
                    jTable1.setValueAt(rez, row, 3);
                    records.add(new RecIntegral(ul, ll, stp, rez));
                });
            } finally {
                // Уменьшаем счетчик задач при завершении
            }
        });
    }
}

```



```

        if (taskCounter.decrementAndGet() == 0) {
            // Все задачи завершены
            System.out.println("All zadachi zavershini");
        }
    }
});
}

long endTime = System.nanoTime();
long duration = endTime - startTime;

double durationInMillis = duration / 1_000_000.0;
System.out.println("Time " + durationInMillis + " milli");

}

private void jButton4ActionPerformed(java.awt.event.ActionEvent evt) {
    tModel.setRowCount(0);

    for (RecIntegral record : records) {
        tModel.addRow(new Object[] {record.getValueUl(), record.getValueLl(),
record.getValueStp(),record.getValueRez()});
    }

}

private void jButton5ActionPerformed(java.awt.event.ActionEvent evt) {
    tModel.setRowCount(0);
    records.clear();
}

/**
 * @param args the command line arguments
 */
public static void main(String args[]) {
    /* Set the Nimbus look and feel */
    //<editor-fold defaultstate="collapsed" desc=" Look and feel setting code (optional) ">
    /* If Nimbus (introduced in Java SE 6) is not available, stay with the default look and feel.
     * For details see http://download.oracle.com/javase/tutorial/uiswing/lookandfeel/plaf.html
     */
    try {
        for (javax.swing.UIManager.LookAndFeelInfo info :
javax.swing.UIManager.getInstalledLookAndFeels()) {
            if ("Nimbus".equals(info.getName())) {

```

```

        javax.swing.UIManager.setLookAndFeel(info.getClassName());
        break;
    }
}
} catch (ClassNotFoundException ex) {
    java.util.logging.Logger.getLogger(ContactEditorUI.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);
} catch (InstantiationException ex) {
    java.util.logging.Logger.getLogger(ContactEditorUI.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);
} catch (IllegalAccessException ex) {
    java.util.logging.Logger.getLogger(ContactEditorUI.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);
} catch (javax.swing.UnsupportedLookAndFeelException ex) {
    java.util.logging.Logger.getLogger(ContactEditorUI.class.getName()).log(java.util.logging.Level.SEVERE, null, ex);
}
//</editor-fold>

```

```

/* Create and display the form */
java.awt.EventQueue.invokeLater(new Runnable() {
    public void run() {
        new ContactEditorUI().setVisible(true);
    }
});
}

```

```

// Variables declaration - do not modify
private javax.swing.JButton jButton1;
private javax.swing.JButton jButton2;
private javax.swing.JButton jButton3;
private javax.swing.JButton jButton4;
private javax.swing.JButton jButton5;
private javax.swing.JLabel jLabel1;
private javax.swing.JLabel jLabel2;
private javax.swing.JLabel jLabel3;
private javax.swing.JScrollPane jScrollPane1;
private javax.swing.JTable jTable1;
private javax.swing.JTextField jTextField1;
private javax.swing.JTextField jTextField2;
private javax.swing.JTextField jTextField3;
// End of variables declaration

```

```

private Double calculateIntegral(Double ll, Double ul, Double stp) {

```

```

int numThreads = 4;
Double n = (ul - ll) / stp;
Double stp_ost = stp * (n - Math.floor(n));

// Создаем потоков
ExecutorService executor = Executors.newFixedThreadPool(numThreads);
List<Future<Double>> futures = new ArrayList<>();

// Разделяем работу на потоки
Double chunkSize = n / numThreads; // Количество шагов на каждый поток
for (int i = 0; i < numThreads; i++) {
    Double start = ll + i * chunkSize * stp; // Начало интервала для потока
    Double end = (i == numThreads - 1) ? ul : start + chunkSize * stp; // Конец интервала для потока

    // Запускаем задачу для каждого потока
    Future<Double> future = executor.submit(() -> calculateChunk(start, end, stp));
    futures.add(future);
}

// Суммируем результаты всех потоков
Double rez = 0.0;
for (Future<Double> future : futures) {
    try {
        rez += future.get(); // Получаем результат каждого потока
    } catch (InterruptedException | ExecutionException e) {
        e.printStackTrace();
    }
}

// Обрабатываем остаточный шаг, если он есть
if (stp_ost > 0) {
    Double osn1 = Math.tan(ul - stp_ost);
    Double osn2 = Math.tan(ul);
    Double h = stp_ost;
    rez += ((osn1 + osn2) * h) / 2;
}

// Завершаем работу потоков
executor.shutdown();
return rez;
}

private static Double calculateChunk(Double start, Double end, Double stp) {

```

```

    Double rez = 0.0;
    for (Double x = start; x < end; x += stp) {
        rez += (Math.tan(x) + Math.tan(x + stp)) * stp / 2;
    }
    return rez;
}

}

```

Выполнение программы

Upper_limit	Lower_limit	Step	Result
0,1	0,1	0,01	0

```

--- exec:3.1.0:exec (default-cli) @ ContactEditor ---
Time 3.4611 milli
All zadachi zavershini

```

Рисунок 1 — Время работы с потоками

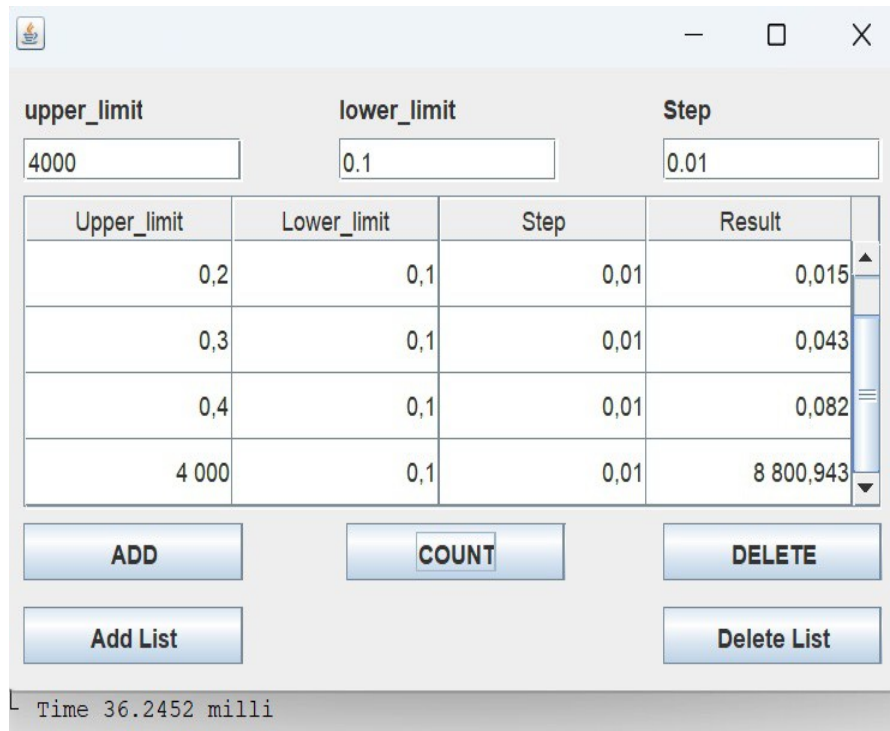


Рисунок 2 — Время работы без потоков

Ход работы

```
long startTime = System.nanoTime();
```

```
records.clear(); // Очищаем список записей
```

```
AtomicInteger taskCounter = new AtomicInteger(i); // Счетчик оставшихся задач
```

```
for (int j = 0; j < i; j++) {
```

```
    final int row = j; // Захватываем текущую строку для использования в потоке
```

```
    // Запускаем асинхронную задачу для каждой строки
```

```
    CompletableFuture.runAsync(() -> {
```

```
        try {
```

```
            Double ul = (Double) jTable1.getValueAt(row, 0);
```

```
            Double ll = (Double) jTable1.getValueAt(row, 1);
```

```
            Double stp = (Double) jTable1.getValueAt(row, 2);
```

```
            Double rez = calculateIntegral(ll, ul, stp); // Вычисляем интеграл
```

```

        // Обновляем таблицу и добавляем запись в records
        SwingUtilities.invokeLater(() -> {
            jTable1.setValueAt(rez, row, 3);
            records.add(new RecIntegral(ul, ll, stp, rez));
        });
    } finally {
        // Уменьшаем счетчик задач при завершении
        if (taskCounter.decrementAndGet() == 0) {
            // Все задачи завершены
            System.out.println("All zadachi zavershini");
        }
    }
});
}

long endTime = System.nanoTime();
long duration = endTime - startTime;

double durationInMillis = duration / 1_000_000.0;
System.out.println("Time " + durationInMillis + " milli");

private Double calculateIntegral(Double ll, Double ul, Double stp) {
    int numThreads = 4;
    Double n = (ul - ll) / stp;
    Double stp_ost = stp * (n - Math.floor(n));

    // Создаем потоков
    ExecutorService executor = Executors.newFixedThreadPool(numThreads);
    List<Future<Double>> futures = new ArrayList<>();

    // Разделяем работу на потоки
    Double chunkSize = n / numThreads; // Количество шагов на каждый поток
    for (int i = 0; i < numThreads; i++) {
        Double start = ll + i * chunkSize * stp; // Начало интервала для потока
        Double end = (i == numThreads - 1) ? ul : start + chunkSize * stp; // Конец интервала для потока

        // Запускаем задачу для каждого потока
        Future<Double> future = executor.submit(() -> calculateChunk(start, end, stp));
        futures.add(future);
    }

    // Суммируем результаты всех потоков

```

```

Double rez = 0.0;
for (Future<Double> future : futures) {
    try {
        rez += future.get(); // Получаем результат каждого потока
    } catch (InterruptedException | ExecutionException e) {
        e.printStackTrace();
    }
}

// Обрабатываем остаточный шаг, если он есть
if (stp_ost > 0) {
    Double osn1 = Math.tan(ul - stp_ost);
    Double osn2 = Math.tan(ul);
    Double h = stp_ost;
    rez += ((osn1 + osn2) * h) / 2;
}

// Завершаем работу потоков
executor.shutdown();
return rez;
}

private static Double calculateChunk(Double start, Double end, Double stp) {
    Double rez = 0.0;
    for (Double x = start; x < end; x += stp) {
        rez += (Math.tan(x) + Math.tan(x + stp)) * stp / 2;
    }
    return rez;
}

```

Пояснение к тексту программы(основные вычисления)

Строки 1-10: Создание счетчика асинхронных задач для работы со строками в таблице.

Строка 11: Вызов метода для вычисления интеграла с помощью нескольких потоков.

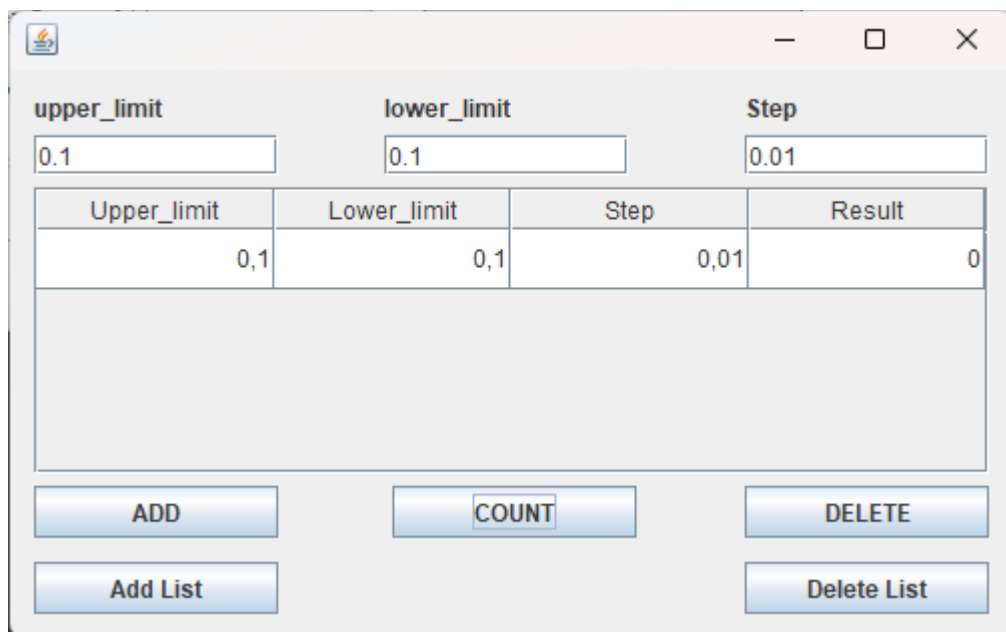
Строка 12-17: Уменьшение счетчика задач на единицу или если задач уже ноль завершение.

Строка 18-21: Вычисление времени работы программы.

Строка 22-28: Создание нескольких потоков

Строка 28-44: Вычисление интеграла в потоках, затем суммирование результатов работы.

Результат выполнения программы



The screenshot shows a window titled "ContactEditor" with a standard Windows-style title bar. Inside the window, there are three input fields at the top: "upper_limit" with the value "0.1", "lower_limit" with the value "0.1", and "Step" with the value "0.01". Below these fields is a table with four columns: "Upper_limit", "Lower_limit", "Step", and "Result". The table contains one row of data with values "0,1", "0,1", "0,01", and "0" respectively. Below the table, there are four buttons: "ADD", "COUNT", "DELETE", and "Add List" (which is positioned below "ADD"), and "Delete List" (which is positioned below "DELETE").

Upper_limit	Lower_limit	Step	Result
0,1	0,1	0,01	0

```
--- exec:3.1.0:exec (default-cli) @ ContactEditor ---  
Time 3.4611 milli  
All zadachi zavershini
```

Рисунок 3 — Результат

Вывод

Изучена работа с потоками. В некоторых задачах лучше не использовать потоки для оптимизации кода.